Task 62: Sort Items by Groups Respecting Dependencies

Problem Description

There are n items each belonging to zero or one of m groups where group[i] is the group that the i-th item belongs to and it's equal to -1 if the i-th item belongs to no group. The items and the groups are zero indexed. Return a sorted list of the items such that: The items that belong to the same group are next to each other in the sorted list. There are some relations between these items where beforeItems[i] is a list containing all the items that should come before the i-th item in the sorted array (to the left of the i-th item). Return any solution if there is more than one solution and return an empty list if there is no solution.

Java Solution

```
class Solution {
    public int[] sortItems(int n, int m, int[] group, List<List<Integer>>
beforeItems) {
        for (int i = 0; i < n; i++) {
            if (group[i] == -1) group[i] = m++;
        }

        List<Integer>[] itemGraph = new ArrayList[n];
        List<Integer>[] groupGraph = new ArrayList[m];
        int[] itemIndegree = new int[n];
        int[] groupIndegree = new int[m];

        for (int i = 0; i < n; i++) itemGraph[i] = new ArrayList<>();
        for (int i = 0; i < m; i++) groupGraph[i] = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            for (int prev : beforeItems.get(i)) {
                itemGraph[prev].add(i);
                itemIndegree[i]++;
                if (group[i] != group[prev]) {
                    groupGraph[group[prev]].add(group[i]);
                    groupIndegree[group[i]]++;
                }
            }
        }

        List<Integer> itemOrder = topoSort(itemGraph, itemIndegree, n);
        List<Integer> groupOrder = topoSort(groupGraph, groupIndegree, m);

        if (itemOrder.isEmpty() || groupOrder.isEmpty()) return new
int[0];

        Map<Integer, List<Integer>> groupedItems = new HashMap<>();
        for (int item : itemOrder) {
            groupedItems.computeIfAbsent(group[item], k -> new
ArrayList<>()).add(item);
        }

        int[] ans = new int[n];
        int idx = 0;
        for (int g : groupOrder) {
            if (groupedItems.containsKey(g)) {
                for (int item : groupedItems.get(g)) {
                    ans[idx++] = item;
```

```
                }
            }
        }
        return ans;
    }

    private List<Integer> topoSort(List<Integer>[] graph, int[] indegree,
int n) {
        List<Integer> res = new ArrayList<>();
        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < n; i++) {
            if (indegree[i] == 0) q.offer(i);
        }
        while (!q.isEmpty()) {
            int u = q.poll();
            res.add(u);
            for (int v : graph[u]) {
                if (--indegree[v] == 0) q.offer(v);
            }
        }
        return res.size() == n ? res : new ArrayList<>();
    }
}
```